Contents

UPDC 1.1.0 Technical Manual (English) **1**
1. Purpose and scope . . . 1
2. Input contract and processing sequence . . . 1
3. Scientific operator and iteration . . . 2
4. Background, DQ, weights, and pedestal . . . 2
5. Target WCS, PSF transport, and polygon geometry . . . 3
6. Numerical precision and performance . . . 3
7. Installation and first run . . . 4
8. Configuration and execution rules . . . 5
9. Cache, builds, and HPC deployment . . . 5
10. Products, validation, and troubleshooting . . . 6
11. Adapter extension and maintenance rules . . . 6
20. License and scientific publication citation . . . 6

UPDC 1.1.0 Technical Manual (English)

Package: `updc-reconstruction` 1.1.0
Numerical core: frozen v17 C implementation
Configuration interface: TOML schema v1
Basis: source code, tests, configuration examples, and validation records in this release. Benchmark numbers describe one host and dataset; they are not runtime guarantees.

1. Purpose and scope

UPDC jointly reconstructs an astronomical image from multiple exposures. Each exposure can have its own geometry, scalar background, pixel weights, and PSF. The reconstruction retains positive and negative target pixels. Detector pixels are modeled through polygon overlap on a common target grid rather than center-point sampling.

The documented end-to-end scientific acceptance is for a JWST/NIRCam workflow. The built-in `fits` adapter accepts normalized FITS data with the contract below; a new instrument still requires independent checks of units, WCS, PSF, DQ semantics, and scientific output. The frozen operator uses one spatially invariant PSF **per exposure**. UPDC does not generate PSFs or produce a covariance-aware post-deconvolution uncertainty image. UPDC is released under the standard MIT License. If UPDC code, its algorithmic implementation, or scientific results produced with UPDC are used in a journal paper, conference proceeding, thesis/dissertation, technical report, data-release paper, or other publication, the UPDC project requires citation of both the 2022 MNRAS method paper and the 2025 ApJS UPdec-Webb paper. See `CITATION.cff`, `CITATION.md`, and `NOTICE`. This scientific-attribution policy is separate from the MIT software license.

Layer	Main files	Responsibility
Stable interface	`updc/cli.py`, `updc/config.py`	TOML, overrides, paths, adapter validation
Instrument adapters	`updc/adapters/`	Instrument defaults and native-data normalization
Orchestration	`updc/_engine.py`	Background, weights, WCS, PSFs, cache, C invocation, FITS
Solver	`updc/c_core/updc_core.c`, `overlap.c`	LinkMaps, FFTs, iterations, diagnostics
PSF projector	`updc/c_core/psf_projector.c`	Polygon flux transport for detector-frame PSFs
Build support	`updc/build.py`	Compiler/FFTW probes and binary signatures

Four C core/geometry files are protected by SHA-256 values in `updc/core_freeze.json`; `updc doctor` and tests verify them. A numerical change needs a new core freeze and scientific regression, not merely revised hashes.

2. Input contract and processing sequence

Each exposure is a same-sized FITS file containing two-dimensional `SCI`, `DQ`, `VAR_POISSON`, `VAR_RNOISE`, and `VAR_FLAT` extensions. Each is paired with one two-dimensional PSF FITS image. `SCI BUNIT` must agree across exposures; primary-HDU `FILTER` and `PUPIL` must also agree. If `BUNIT` is absent, UPDC labels output SCI with the default `MJy/sr`; generic FITS inputs should therefore provide their real unit explicitly. `fits` mode reads a

celestial FITS WCS from the `SCI` header. `gwcs` mode reads JWST `meta.wcs`. SCI, background, and variance must have consistent physical units; PSFs must be nonnegative and normalizable.

```
FITS exposures + one PSF per exposure
  → pairing, dimension/unit checks, DQ/background/weight preparation
  → common linear TAN target WCS
  → WCS mapping of detector-pixel corners
  → PSF projection, optional crop, normalization
  → polygon LinkMaps, initial estimate, sensitivity denominator
  → signed-x iterations with FreezeMu
  → SCI/WHT/diagnostic FITS and optional metrics JSON
```

Pairs can be selected by numeric filename IDs, sorted order, or CSV manifest. `pairing="index"` extracts IDs from names such as `..._00001_nrca1_cal.fits` and `..._1.fits`. `sorted` pairs lexicographically and must be checked for accidental mismatches. A manifest makes the relationship explicit:

```
exposure,psf,psf_x,psf_y
data/exp1.fits,psfs/psf1.fits,1023.5,1023.5
data/exp2.fits,psfs/psf2.fits,1023.5,1023.5
```

Paths in a manifest are relative to its directory. Optional `psf_x`/`psf_y` columns provide a detector anchor for each exposure and must appear together. Without them the detector center is used. Both anchor and entire detector-frame PSF support must lie inside the detector.

3. Scientific operator and iteration

For exposure k, the forward operator is $H_k = S_k P_k$. P_k convolves on the target grid with that exposure's PSF. S_k integrates the convolved image over mapped detector pixels. For detector pixel p and target pixel i,

$$S_{k,pi} = A_{k,pi}/J_{k,p},$$

where $A_{k,pi}$ is their polygon-overlap area and $J_{k,p}$ is the **full** area of the mapped detector quadrilateral. Clipping is exact for the straight-sided quadrilateral constructed from mapped corners; it is not analytic integration of continuously curved WCS behavior within a pixel. Overlap at an image boundary still divides by the full $J_{k,p}$, so edge pixels are not artificially renormalized.

Let $y_k = SCI_k$, b_k be an exposure's scalar background, $p_k \geq 0$ its positivity pedestal, and W_k its detector weight. The Python driver **does not pre-subtract** b_k from SCI. For usable detector pixels,

$$y_{k,\mathrm{eff}} = y_k + p_k, \qquad \mu_k = H_k x + b_k + p_k.$$

An unfrozen pixel with $\mu_k > 0$ uses $r_k = y_{k,\mathrm{eff}}/\mu_k$. If $\mu_k \leq 0$ at any iteration, **FreezeMu** permanently sets $r_k = 1$ for that pixel. The pixel is not discarded: its weight still contributes to the scatter. The logged `1e-12` value is only a diagnostic marker, never a replacement divisor.

The multiplicative update is

$$D = \sum_k H_k^T W_k, \qquad x^{(n+1)} = x^{(n)} \odot \frac{\sum_k H_k^T (W_k \odot r_k^{(n)})}{D}.$$

D is computed once before iterations. Target pixels with $D \leq 10^{-8}$ are set to zero. The initial $x^{(0)}$ is a signed, weighted drizzle of raw SCI with the corresponding weighted scalar background removed. The solver does **not** clip negative x values. It sets only tiny negative multiplicative factors caused by roundoff to zero and fails on substantially negative factors. This weighted signed update should not be interpreted as an ordinary nonnegative Poisson Richardson–Lucy solver.

4. Background, DQ, weights, and pedestal

Preparation starts from finite SCI and required variance values and retains pixels satisfying `(DQ & bad_dq_mask)==0`. The JWST mask default is `531=1+2+16+512`; the generic FITS example uses `0`, since bit meanings are instrument dependent. Scalar sky background uses up to six asymmetric clipping passes. Each

pass estimates median and MAD-based scale, then keeps candidate sky pixels in `[median−4.5σ, median+2.5σ]` to suppress positive sources. A standard-deviation fallback handles zero MAD; too few pixels or no positive scale raises an error.

`weight_mode`	Raw weight at a usable pixel	Interpretation
`rnoise-sky` (default)	$1/(VAR_RNOISE + \text{median}_{sky}\, VAR_POISSON)$	Keeps instrumental variation without
`totalvar`	$1/(VAR_POISSON + VAR_RNOISE + VAR_FLAT)$	Uses all three pixelwise variance plan
`mask`	1	Validity mask only

Invalid or nonpositive weights become zero. The current implementation still computes a usable sky `VAR_POISSON` statistic in `mask` mode. Weights are scaled internally for numerical conditioning and restored for output WHT.

The automatic pedestal makes `SCI+p` positive at every positive-weight pixel. With minimum usable SCI m, sky scale σ, background b, and requested sigma floor s (`poisson_pedestal_sigma`),

$$p_{auto} = \max(0, \max(10^{-12}, 10^{-6}\sigma) - m), \quad p_{request} = \max(0, s\sigma - b), \quad p = \max(p_{auto}, p_{request}).$$

This is a **constant offset of observed and modeled means**, not a random noise realization; it does not change weights. Usable raw `SCI<=0` pixels remain in the fit.

5. Target WCS, PSF transport, and polygon geometry

The target WCS takes CRVAL and the first-order pixel matrix from the first `SCI` celestial FITS WCS, divides pixel scale by `upsample`, and constructs a TAN grid without high-order target distortion. UPDC samples 65 points along each edge of each exposure, maps them with the requested WCS, and bounds the result with a four-target-pixel margin. It then maps the $(H+1) \times (W+1)$ detector-pixel corner lattice for each exposure. `gwcs` requires JWST `meta.wcs`; `fits` uses `SCI` FITS WCS; `auto` warns and falls back to FITS if GWCS is unavailable. Non-JWST data should request `fits` explicitly.

A detector-frame PSF is first oriented into detector x/y axes using `psf_orientation`. It must have odd dimensions and finite, nonnegative values. For width `PW`, height `PH`, oversampling s, and detector anchor (u_0, v_0), its **pixel boundaries** are

$$u_j = u_0 + (j - PW/2)/s, \qquad v_i = v_0 + (i - PH/2)/s.$$

Each boundary corner travels through that exposure's detector-to-sky WCS and inverse target WCS. Subtracting the mapped anchor removes dither translation while retaining rotation, scale, shear, and distortion. **Exposure 1 is transformed as well.** The C projector transfers each input PSF pixel's flux to an odd, symmetric target support using `overlap_area/full_mapped_quad_area`. A projection relative flux error above `5e-5` fails.

`psf_crop_size` applies an optional odd square **after** projection; `0` disables it. The JWST default is `1431`, which requires a mapped PSF at least that large. Discarding more than `0.1%` of mapped flux warns. The retained PSF is subsequently normalized by the solver. For generic FITS, an omitted crop value defaults to `0` so small PSFs work. `psf_frame="target"` bypasses mapping and this crop; its input must already be on the target grid, with any requested transpose handled by the core.

6. Numerical precision and performance

Mapped vertices, clipping tests, and intersections use float64. Polygon areas are calculated after translation to local coordinates using float32/FMA. SCI, WHT, PSF normalization, FFTW3f convolution, and solver arithmetic use float32. FFTW plan, padding, worker topology, and reduction order may cause expected float32-level differences; cross-platform qualification should compare tolerances and diagnostics rather than bitwise equality alone.

For each exposure, a sparse CSR `LinkMap` holds offsets, target indices, and A/J fractions. ABI v2 independently selects `DELTA16` (32-bit per-pixel base plus 16-bit link deltas) when every actual index span fits; otherwise it uses `ABS32`. The `regular-3x3` value in logs is diagnostic only. LinkMaps and PSF spectra can be reused through memory mapping.

FFTW3f uses padding sufficient for linear convolution (`target + PSF - 1`). Each iteration performs one FFT of x, N forward-model inverse FFTs, N scatter FFTs, and one combined adjoint inverse FFT: **`2N+2` transforms**. The denominator uses the same frequency-domain reduction. A fused detector loop computes prediction, FreezeMu, weighted ratio, and scatter while links are resident in cache. A persistent exposure worker pool serves all iterations.

Parameter	Effect
`fft_workers`	Threads within each FFTW transform
`exposure_workers`	Exposures processed concurrently, capped by exposure count
`preprocess_workers`	Parallel Python/C preprocessing
`workers`	Compatibility default for FFT workers; stable config also supplies explicit `fft_workers`
`fft_plan`	`estimate` plans quickly; `measure` can improve repeated execution
`fft_autotune`, `retune_fft`	Benchmark/cache or rebenchmark padded FFT shapes

Nested parallelism can reach roughly `fft_workers × exposure_workers`; benchmark on the allocated host rather than setting every control to the full core count. The recorded EPYC 7763 acceptance used nine 2048^2 exposures, a 4565×4530 target, 1431^2 PSFs, 19 iterations, 6000^2 FFT, and 9/9/9 workers. Warm-cache solver time was **21.207 s**, total time **23.196 s**, peak RSS **9.189 GiB**, and iteration transforms **380**. This is a regression reference, not a prediction for another system.

7. Installation and first run

UPDC requires Python **>=3.10**, NumPy, Astropy, a C11 compiler, and single-precision FFTW3 development files including its thread library. JWST/GWCS processing also needs the `jwst` extra. On Ubuntu/Debian install `libfftw3-dev pkg-config cmake build-essential`; on Homebrew install `fftw pkg-config cmake`; on Conda use conda-forge `fftw pkg-config`. Then install the wheel or run `python -m pip install .` from this directory; JWST users may run `python -m pip install '.[jwst]'`.

```
updc doctor
updc init updc.toml
# edit real input, PSF, and output paths and scientific settings
updc validate updc.toml
updc updc.toml
```

`updc init` writes a **JWST template** whose WCS, globs, orientation, and 1431 crop are explicitly set. Merely changing `adapter` to `fits` leaves those explicit values in place. For generic FITS, start from `examples/generic_fits.toml`, edit paths, and verify pairing, `BUNIT`, PSF orientation/sampling, and crop. `updc config.toml` is shorthand for `updc run config.toml`; `python -m updc` also works. `run_updc.py` and `updc-jwst` preserve the older flag interface; small PSFs there need explicit `--psf-crop-size 0`.

A generic FITS configuration using the `pairs.csv` above can start as follows; set PSF oversampling to match the actual file:

```
[updc]
schema_version = 1
[instrument]
adapter = "fits"
[input]
manifest = "./pairs.csv"
wcs_mode = "fits"
bad_dq_mask = 0
[psf]
psf_frame = "detector"
psf_orientation = "as-is"
psf_oversample = 2
psf_crop_size = 0
[solver]
iterations = 19
upsample = 2
save_every = 0
```

```toml
[output]
output_dir = "./result"
```

8. Configuration and execution rules

TOML relative paths resolve against the TOML directory; relative paths inside a manifest resolve against the manifest directory. Unknown top-level sections and keys are rejected. `[instrument.options]` is reserved for adapters. `updc validate` checks configuration, path roles, adapter requirements, and frozen core hashes; it does **not** inspect every FITS pixel or substitute for scientific acceptance. `--set section.key=TOML-value` temporarily overrides a configured value, and adapters preserve that explicit override:

```
updc updc.toml --set solver.iterations=39 --set performance.fft_workers=4
updc updc.toml --set 'psf.psf_crop_size=0'
```

Section	Important fields and constraints
`[updc]`	`schema_version=1`
`[instrument]`	`adapter="jwst"`, `"fits"`, or installed plugin name; optional adapter `options`
`[input]`	At least one of `input_dir`, `manifest`, `jwst_dir`; globs, `pairing`, `wcs_mode`, `weight_mode`, `bad_dq_mask`, `poisso`
`[psf]`	`psf_dir` unless supplied by manifest/JWST directory; glob, orientation, frame, oversampling, crop, paired
`[solver]`	Positive `iterations`, `upsample`; `save_every=0` writes only final, otherwise every Nth plus final
`[performance]`	Positive worker counts; `fft_plan="estimate"/"measure"`, Boolean autotune/retune
`[cache]`	`enabled`, paths, `reset`, nonnegative `prune_after_days`
`[output]`	`output_dir`, `output_stem`, `metrics_json`, `keep_work`, `work_dir`
`[build]`	`cc`, optional prebuilt `c_core_binary` and `psf_projector_binary`

General defaults include `iterations=19`, `upsample=2`, `save_every=5`. JWST defaults include `wcs_mode="gwcs"`, `psf_orientation="stored-transposed"`, and `psf_crop_size=1431`; generic FITS uses `fits`, `as-is`, and `0`. If omitted, `psf_oversample` equals `upsample`. Choose scientific settings from the actual data and PSF rather than treating example numbers as universal.

9. Cache, builds, and HPC deployment

The default dataset cache root is `~/.cache/updc-jwst/data`; global FFTW wisdom defaults to `~/.cache/updc-jwst/fftwf_wisdom`. The Python preprocessing ABI is 1 (marker `updc-v16-preprocess-1`), LinkMap ABI is 2 (`ULM2`), and FFT-autotune ABI is 1. Reusable files include SCI/WHT working planes, target WCS, PSFs, vertices, LinkMaps, initial estimate, denominator, and spectra. Raw iterations and diagnostic CSVs belong to a per-run temporary directory. An explicit `work_dir` or `keep_work` retains intermediates and bypasses the normal dataset-cache hit path.

The dataset key is the first 24 hex characters of SHA-256 over a description containing input/PSF **paths, file sizes, mtime_ns**, and processing settings. It is **not a byte-for-byte content hash**. If an external tool changes a file while preserving size and mtime, use `reset="current"` or a new cache root to avoid reusing stale preprocessing.

`cache.reset="current"` deletes only this dataset's cache; `"all"` deletes dataset directories with 24-hex-character names under the root. Reset is evaluated **every run**, so return it to `"none"`. `prune_after_days>0` removes stale dataset caches at the next start; `0` disables pruning. Neither action is a background service or removes global wisdom outside that root. `cache.enabled=false` uses temporary work and ignores reset/prune. RAM, mmaps, and FFTW workspaces are released at process exit; the OS reclaims filesystem page cache as needed.

Without prebuilt binaries, UPDC compiles the frozen C sources at runtime. The build helper probes C11 atomics, native CPU flags (`-march=native`, or `-mcpu=native` on macOS ARM), and FFTW3f thread linkage; it filters HPC C++ include paths that can shadow C's `stdatomic.h`. A compiled-binary cache signature includes source hash, compiler path/version, flags, FFTW details, and platform. If pkg-config metadata is absent, set `UPDC_FFTW_INCLUDE` and `UPDC_FFTW_LIB`. On clusters, load modules, run `updc doctor --cc gcc`, then benchmark worker combinations under the job's CPU and memory allocation. Prebuilt binaries must match this frozen core and deployment environment.

10. Products, validation, and troubleshooting

Output names look like `updc_<filter>_<detector>_iter05_i2d.fits`. Each file contains primary, `SCI`, `WHT`, `UPDC_BKG`, and `UPDC_PSFGEOM` HDUs. After writing, UPDC checks finite SCI, finite nonnegative WHT, diagnostic table lengths, and FITS checksums. `SCI` is the signed reconstruction x. `WHT=\sum_k S_k^T W_k` is the **pre-PSF coverage/weight map**: it is neither iteration denominator D nor a rigorous inverse variance including deconvolution covariance. WHT has unit `1` in `mask` mode; otherwise its header uses the squared inverse of input `BUNIT`.

`UPDC_BKG` records scalar background, noise, pedestal, and retained nonpositive SCI counts. `UPDC_PSFGEOM` records anchors, projected/cropped dimensions, flux loss, and WCS used. Primary/SCI `MUFRTOT` and `MUFRNEW` report FreezeMu counts at that iteration. Optional `metrics_json` adds input statistics, per-exposure/per-iteration FreezeMu, cache hit information, PSF geometry, stage timings, and output paths. `write_transformed_psf=true` additionally emits target-frame PSFs for inspection.

For a new dataset, use this acceptance sequence:

1. Run `updc doctor` and `updc validate`; confirm dependencies, core hashes, and real paths.
2. Inspect exposure/PSF pairs, SCI units, DQ bits, FITS/GWCS mapping, PSF axes/anchors, and flux retention in `UPDC_PSFGEOM`.
3. Run a few iterations and inspect sky/background, WHT coverage, negative-pixel counts, FreezeMu growth, and target WCS.
4. Compare SCI/WHT max and RMS differences, sums, negative counts, and diagnostics with a trusted reference; `python tools/compare_fits.py reference.fits candidate.fits` helps.
5. Once scientific settings are fixed, benchmark worker counts and warm-cache runs; archive configuration, dependency versions, metrics, and logs.

Run code checks with `python -m pytest -q`. Build and self-test C with `cmake -S updc/c_core -B build -DUPDC_NATIVE_ARCH=OFF`, `cmake --build build`, and `ctest --test-dir build --output-on-failure`. A successful `updc validate` establishes interface/path validity, not scientific correctness.

For common failures: if crop 1431 exceeds a small mapped PSF, choose an appropriate odd crop or `0`; if GWCS is unavailable, install the JWST extra or use `fits` for compatible FITS-WCS data; for `stdatomic.h` or FFTW failures, inspect `updc doctor` and cluster modules; if edited inputs appear to reuse old preprocessing, check path/mtime and rebuild the current cache. Detailed background is in `ARCHITECTURE.md`, `PSF_GWCS_POLYGON.md`, `CACHE_ABI.md`, `HPC_LINUX.md`, and `REFERENCE_BENCHMARK.md`.

11. Adapter extension and maintenance rules

A third-party adapter subclasses `updc.adapters.InstrumentAdapter`, implements `apply_defaults(config)` and `validate(config)`, and may normalize native files into the FITS contract in `prepare(config, context)`, returning `PreparedRun(config=updated_config)`. Temporary files belong under `context.staging_dir`, which is cleaned after the run. Register the adapter in the `updc.instrument_adapters` Python entry-point group; `updc adapters` then lists it. See `examples/custom_adapter.py` and `docs/ADAPTERS.md`.

Adapters handle ingestion, validation, and normalization. A spatially varying PSF or genuinely different forward operator requires a newly designed, validated, and frozen numerical core. A persistent format change requires cache ABI review and increment where incompatible. Before distribution, run Python tests, C self-test, clean-environment wheel installation, representative real-data regression, and license/release-manifest review.

20. License and scientific publication citation

UPDC 1.1.0 is distributed under the standard MIT License in the repository-root `LICENSE`.

For journal papers, conference proceedings, theses/dissertations, technical reports, data-release papers, or other publications that use UPDC code, its algorithmic implementation, or scientific results produced with UPDC, the UPDC project requires citation of **both** publications:

1. Wang, L., Li, G., & Kang, X. 2022, *Towards super-resolution via iterative multi-exposure coaddition*, MNRAS, 517, 787, doi:10.1093/mnras/stac2664.
2. Wang, L., Shan, H., Nie, L., et al. 2025, *UPdec-Webb: A Data Set for Coaddition of JWST NIRCam Images*, ApJS, 276, 36, doi:10.3847/1538-4365/ad9566.

Machine-readable citation metadata and BibTeX are provided in `CITATION.cff` and `CITATION.md`. This publication-citation policy is a scientific attribution policy of the UPDC project. It is separate from, and does not modify, the MIT License.